Laboration: icke-linjära ekvationer

1 Kort introduktion

Icke-linjära ekvationer dyker upp i många tillämpningar. I allmänhet kan icke-linjära ekvationer inte lösas analytiskt, då använder man i stället numeriska lösningsmetoder.

- Vi ska här studera så kallade *iterativa metoder* för att lösa icke-linjära ekvationer. Grundidén bakom dessa metoder är att man först *gissar* ett nollställe eller ett intervall där man tror att ett nollställe finns, och sen successivt förbättrar gissningen tills ett tillräckligt noggrant approximativt nollställe har hittats. Varje "varv" i en sådan algoritm, där gissningen förbättras, kallas för en *iteration*.
- Det finns många olika metoder för att lösa icke-linjära ekvationer numeriskt. Två enkla, men vanliga, iterativa metoder är *Newton-Raphsons metod* och *bisektionsmetoden*. Vi ska här titta närmare på hur dessa två metoder fungerar.

Börja med att ladda ner Python-filerna för Sektion 2 av denna laboration: **Newtondemo.py** och **Bisektiondemo.py**.

Obs! Om du använder en IDE så kan du behöva ställa in så att interaktiva figurer kan skapas. Till exempel i Spyder: Preferences → IPython Console → Graphics → Backend → Automatic.

2 Newton-Raphsons metod och bisektionsmetoden

Vi ska börja med att studera den icke-linjära ekvationen

$$x^2 = 4\sin(x) + 1. \tag{1}$$

De iterativa metoderna vi ska undersöka används för att hitta nollställen av en funktion, alltså måste ekvationen skrivas om på formen $f(x) = 0$. I det här fallet får vi

$$f(x) = x^2 - 4\sin(x) - 1. \tag{2}$$

1. Kör programmet `Newtondemo` och experimentera med olika val av startgissning. Välj startgissning så att du i något fall hittar det ena nollstället och i något fall det andra. Var uppmärksam på startgissningens inverkan på antalet iterationer. När Newton-Raphson väl hamnat "tillräckligt nära" ett nollställe minskar felet väldigt snabbt. Kan du se någon princip för hur antalet nollor i felet ökar (ungefär) i varje iteration?

2. Newton-Raphsons metod bygger på att nästa gissning ges av nollstället till tangenten i nuvarande punkt. Försök se det i figuren, dvs. hur Newton-Raphsons metod fungerar rent principiellt. Det här kan leda till problem när tangenten pekar långt ifrån nollstället. Testa startgissningar kring $x = 1$. Vad händer?

3. Kör programmet `Bisektiondemo` och experimentera med olika val av startintervall. Välj startintervall så att du i något fall hittar det ena nollstället och i något fall det andra. Observera att man kan välja stora intervall, förutsatt att intervallet enbart täcker ett nollställe. Newton-Raphson gick väldigt snabbt mot nollstället när den väl hamnat "tillräckligt" nära. Men hur är det med bisektionsmetoden? Vilken metod verkar vara snabbast?

4. Bisektionsmetoden bygger på successiv halvering av intervallet på sådant sätt att att funktionsvärdena i intervallsgränserna hela tiden har olika tecken. På så sätt finns alltid minst en rot i intervallet, vars längd halveras i varje iteration. Varför fungerar inte bisektionsmetoden om startintervallet innefattar två nollställen? Eller inget nollställe? Vad skulle hända om ett startintervall med tre rötter valdes?

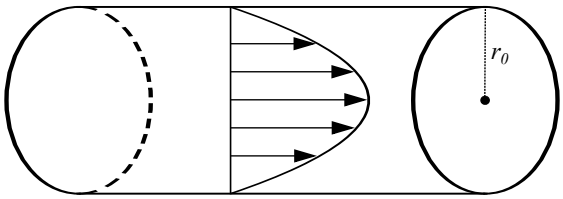
3 SciPy-funktioner för lösning av icke-linjära ekvationer

När man löser icke-linjära ekvationer i Python så kan man använda funktioner från SciPy. Den här delen av laborationen är en introduktion till en dessa funktioner, `brentq`, avsedd för icke-linjära ekvationer med en obekant (skalära ekvationer). Den importeras enligt:

```
1  from scipy.optimize import brentq
```

Funktionen bygger på en algoritm som kombinerar flera metoder, bl.a. bisektionsmetoden och en variant av Newton-Raphsons metod (som kallas sekantmetoden, och som inte förutsätter tillgång till derivatan av den funktion man söker nollställe till). Likt bisektionsmetoden så kräver `brentq` ett startintervall där funktionen byter tecken.

Vi ska här utgå från problemställningen i laborationen om integrering, nämligen hastigheten hos en flödande vätska i ett rör med radie r_0, se Figur 1. Hastigheten v vid



Figur 1: Hastighetsfördelning i cylindriskt rör med radie r_0.

avstånd r från mittpunkten beskrivs av den matematiska modellen

$$v(r) = 2(1 - \frac{r}{r_0})^{1/6}. \tag{3}$$

Vi är intresserade av vid vilken radie r som hastigheten är c, dvs. problemet är att hitta det värde på r så att $v(r) = c$. Detta är en icke-linjär ekvation som lämpligen löses med hjälp av `brentq`.

5. Funktionen `brentq` används för att hitta rötter, första steget blir alltså att skriva om ekvationen till ett nollställesproblem. Dvs. hitta en funktion $f(r)$ så att lösningen till ekvationen $f(r) = 0$ är samma som lösningen till $v(r) = c$.

6. När man löser skalära icke-linjära ekvationer är det ofta användbart att undersöka problemet grafiskt, på så sätt får man en uppfattning om hur många nollställen det verkar finnas och vad som kan vara en bra startgissning eller ett bra startintervall. Börja med att definiera $f(r)$ som en Python-funktion, använd strukturen

```
1  def f(r):
2    return ...
```

   Implementera funktionen så att den kan anropas med `r` som en NumPy-vektor och returnera $f(r)$ för alla värden i vektorn. Använd $r_0 = 3$ och $c = 1.8$. Plotta

därefter funktionen med förslagsvis `matplotlib`. Tänk på att att radien r inte kan vara mindre än 0 eller större än r_0, vi är alltså intresserade av $f(r)$ på intervallet $r \in [0, r_0]$. Hur många nollställen verkar det finnas?

7. Lös den icke-linjära ekvationen med hjälp av `brentq` genom anropet

```
root = brentq(f,xl,xr)
```

   där `f` är Pythonfunktionen och de två sista argumenten definierar startintervallet. Roten lagras i variabeln `root`. Fundera på vad som är ett lämpligt startintervall, utgå från figuren. Verifera att funktionen har löst ekvationen rätt genom att studera figuren eller anropa `f(root)`.

8. I det här fallet vore det bra om vi på ett enkelt sätt kunde variera parametrarna r_0 och c när vi använder `brentq`. Dessa parametrar måste då skickas in i `brentq` och i sin tur vidare in i `f`. Ändra funktionen `f` så att r_0 och c skickas in som argument, funktionsstrukturen blir

```
def f(r,r0,c):
  return ...
```

   Testa att plotta funktionen som tidigare med några olika val av r_0 och c så att du vet att den funkar. För att använda `brentq` då funktionen `f` har fler än ett argument används nyckelordet `args`, anropet blir

```
root = brentq(f,xl,xr,args=(r0,c))
```

   Det är nu enkelt att ändra r_0 och c till andra värden, sedan plotta och hitta nollstället igen. Gör det för några olika val av r_0 och c.

9. Slutligen, sätt $r_0 = 3$ och $c = 4$ och försök lösa ekvationen med hjälp av `brentq`. Vad händer och varför? Studera figuren.

Det finns ett flertal funktioner i SciPy för att lösa icke-linjära ekvationer, se onlinedokumentationen på `https://docs.scipy.org/doc/scipy/reference/optimize.html`. Om du blir inspirerad att göra mer så kan du ersätta `brentq` i din kod med en annan SciPy-funktion, till exempel `toms748`. Läs dokumentationen för att förstå hur de olika funktionerna ska anropas.

4 Beräkning av andel vattenånga

Nu är du klar med studierna och kommer till din första arbetsplats. Där blir du placerad i en grupp som arbetar med en kemiteknisk process där vattenånga upphettas till så höga temperaturer att vattenmolekyler börjar dissociera (sönderdelas) i syre (O_2) och väte (H_2) enligt processen

$$H_2O \rightleftharpoons H_2 + \frac{1}{2}O_2. \tag{4}$$

Reaktionen äger rum under trycket $p = 3$ atmosfärer och med jämviktskonstanten $K = 0.05$. Dina kollegor och du diskuterar hur stor andel av vattenmolekylerna som kommer att sönderdelas. En av kollegorna bläddrar i litteraturen och hittar en matematisk modell för hur andelen molekyler som sönderdelas, x, beror på trycket och

reaktionens jämviktskonstant. Enligt modellen är sambandet

$$K = \frac{x}{1-x}\sqrt{\frac{2p}{2+x}}. \tag{5}$$

Problemet är att det inte går att lösa ut x ovanb.[1]

10. Skriv ett Python-program som besvarar ovanstående problem, dvs. löser den ickelinjära ekvationen (5) med $p = 3$ och $K = 0.05$. Använd föregående del av laborationen som mall. Skriv om till ett nollställesproblem, plotta funktionen och anropa `brentq`. Fundera på vad x står för, och därmed vilket intervall som är fysikaliskt rimligt när man plottar och väljer startintervall.

[1] Problemformuleringen är baserad på problem 6.14 i Steven C. Chapra, *Applied Numerical Methods with MATLAB for Engineers and Scientists, 4th Edition*, McGraw-Hill, 2018, s 191.